UTEC
Universidad Tecnológica

Patrón Builder

El **Patrón Builder** es un patrón de diseño creacional que se utiliza para construir objetos complejos de manera controlada, paso a paso. A diferencia de otros patrones de creación, el Builder permite la creación de un objeto en varias etapas, lo que facilita la creación de objetos que requieren múltiples configuraciones o que tienen una gran cantidad de parámetros opcionales.

Objetivo

El objetivo principal del patrón Builder es separar la construcción de un objeto complejo de su representación, de manera que el mismo proceso de construcción pueda crear diferentes representaciones. Este patrón es útil cuando los objetos a construir tienen muchos parámetros, algunos de los cuales pueden ser opcionales o tener valores predeterminados.

Cómo Funciona

1. **Director**: Opcionalmente, se puede tener una clase director que gestiona el proceso de construcción utilizando el Builder.
2. **Builder**: Esta es una interfaz o clase abstracta que define los pasos para construir el objeto. Cada paso construye una parte del objeto.
3. **Concrete Builder**: Implementa la interfaz del Builder, proporcionando implementaciones específicas para los pasos de construcción. También contiene una referencia al objeto que se está construyendo.
4. **Producto**: Es el objeto final que se construye, el cual es retornado una vez completado el proceso de construcción.

Ventajas del Patrón Builder

1. **Control sobre la construcción**: Permite la construcción paso a paso de un objeto, lo que es ideal para objetos que requieren configuraciones específicas o que tienen múltiples parámetros opcionales.
2. **Código más limpio**: Evita la creación de constructores con una gran cantidad de parámetros, lo que puede ser confuso y propenso a errores.
3. **Reutilización**: Se pueden reutilizar diferentes Builders para crear diferentes representaciones del objeto complejo.
4. **Flexibilidad**: El mismo Builder puede usarse para crear distintas versiones de un objeto complejo mediante la combinación de diferentes pasos.

Ejemplo en Java

UTEC
Universidad Tecnológica

Imaginemos que estamos creando un objeto **"Casa"** que tiene varios atributos como `puertas`, `ventanas`, `garaje`, `piscina`, etc. Algunos de estos atributos son opcionales, y otros dependen de otros atributos (por ejemplo, un garaje solo es útil si hay un coche).

1. Clase Producto

```
public class Casa {
    private int puertas;
    private int ventanas;
    private boolean garaje;
    private boolean piscina;

    // Constructores y métodos de acceso
    public Casa(int puertas, int ventanas, boolean garaje, boolean
piscina) {
        this.puertas = puertas;
        this.ventanas = ventanas;
        this.garaje = garaje;
        this.piscina = piscina;
    }

    @Override
    public String toString() {
        return "Casa [puertas=" + puertas + ", ventanas=" + ventanas +
               ", garaje=" + garaje + ", piscina=" + piscina + "]";
    }
}
```

2. Interfaz del Builder

```
public interface CasaBuilder {
    CasaBuilder setPuertas(int puertas);
    CasaBuilder setVentanas(int ventanas);
    CasaBuilder setGaraje(boolean garaje);
    CasaBuilder setPiscina(boolean piscina);
    Casa build(); // Construye y retorna la casa final
}
```

UTEC
Universidad Tecnológica

3. Implementación del Concrete Builder

```
public class CasaConcretaBuilder implements CasaBuilder {
    private int puertas;
    private int ventanas;
    private boolean garaje;
    private boolean piscina;

    @Override
    public CasaBuilder setPuertas(int puertas) {
        this.puertas = puertas;
        return this;
    }

    @Override
    public CasaBuilder setVentanas(int ventanas) {
        this.ventanas = ventanas;
        return this;
    }

    @Override
    public CasaBuilder setGaraje(boolean garaje) {
        this.garaje = garaje;
        return this;
    }

    @Override
    public CasaBuilder setPiscina(boolean piscina) {
        this.piscina = piscina;
        return this;
    }

    @Override
    public Casa build() {
        return new Casa(puertas, ventanas, garaje, piscina);
    }
}
```

4. Uso del Builder

UTEC
Universidad Tecnológica

```
public class ConstructorDeCasas {
    public static void main(String[] args) {
        CasaBuilder builder = new CasaConcretaBuilder();

        // Crear una casa con garaje y piscina
        Casa casa1 = builder.setPuertas(4)
                            .setVentanas(10)
                            .setGaraje(true)
                            .setPiscina(true)
                            .build();

        System.out.println(casa1);

        // Crear una casa sin garaje ni piscina
        Casa casa2 = builder.setPuertas(3)
                            .setVentanas(6)
                            .setGaraje(false)
                            .setPiscina(false)
                            .build();

        System.out.println(casa2);
    }
}
```

Explicación

En este ejemplo, usamos el patrón Builder para construir diferentes instancias del objeto **Casa**. Podemos personalizar cada casa según nuestras necesidades (con o sin garaje, con más o menos puertas, etc.), y al final de la construcción, obtenemos una instancia del objeto **Casa** con las configuraciones que especificamos. Esto nos proporciona un método flexible y limpio para construir objetos complejos sin necesidad de utilizar constructores largos y difíciles de mantener.

Conclusión

El patrón Builder es ideal para situaciones donde se requiere la creación de objetos complejos con múltiples configuraciones opcionales. Permite un control detallado sobre la creación del objeto y asegura que el código se mantenga limpio y legible.